BharatRWA

Problem Statement -

Real-world assets such as gold, real estate, and commodities represent trillions of dollars in value globally. However, access to these assets is often limited due to high investment barriers, regulatory complexity, and lack of liquidity.

Traditional financial systems rely on centralized intermediaries such as banks, custodians, and brokers to manage asset ownership and transactions. These intermediaries increase operational costs, reduce transparency, and limit accessibility for retail investors.

Blockchain technology introduces the concept of tokenisation, where physical assets can be represented as digital tokens on a distributed ledger. Tokenisation enables fractional ownership, transparent transaction records, and automated compliance through smart contracts.

However, current tokenisation platforms face significant challenges, particularly in the areas of regulatory compliance, investor identity verification, and privacy protection.

Problems in Existing Systems -

1. High Investment Barriers
2. Lack of Liquidity
3. Limited Transparency
4. Privacy Risks in Compliance
5. Limited Global Accessibility

Solution -

BharatRWA, introduces a blockchain-based infrastructure for compliant tokenisation of real-world assets.

Key innovations include:

- Tokenisation of assets into ERC20 fractional ownership tokens
- Zero-Knowledge KYC verification to protect investor privacy
- Smart contract-based compliance enforcement
- Transparent asset registry linked to custodians
- Oracle-based asset price feeds
- Automated dividend distribution mechanisms

This system aims to create a secure, transparent, and globally accessible infrastructure for real-world asset investment.

Real Life Use Cases of BharatRWA -

1. Gold Fractional Ownership - Gold is traditionally expensive for retail investors. BharatRWA allows fractional token ownership of physical gold stored by a custodian.
    - Process -
        - ☐ Custodian deposits verified gold.
        - ☐ Asset is registered on-chain.
        - ☐ ERC20 tokens representing gold shares are minted.
        - ☐ Investors buy tokens.
        - ☐ Investors receive proportional value when price increases.
    - Benefits -
        - ☐ Low entry barrier
        - ☐ High liquidity
        - ☐ Transparent ownership
        - ☐ On-chain verification
2. Real Estate Fractional Investment - High-value properties can be tokenised. Example: A ₹5 Crore property can be divided into 500,000 tokens.
    - Investors buy tokens and earn:
        - ☐ Rental income
        - ☐ Property appreciation
    - Benefits:
        - ☐ No paperwork
        - ☐ Global investor access
        - ☐ Liquidity compared to traditional property markets
3. Commodity Tokenisation - Assets like:
    - Gold
    - Silver
    - Oil
    - Agricultural commodities can be tokenised and traded on-chain. Investors gain exposure without physical storage or logistics.
4. Private Asset Tokenisation - Startups and private funds can tokenize
    - Infrastructure projects
    - Renewable energy plants
    - Private equity

5. Cross-border Investment - Using blockchain
    - Investors globally can participate
    - Compliance enforced via ZK-KYC
    - Settlement occurs instantly
6. DeFi Collateralisation- Tokenised RWAs can be used as collateral in DeFi.
   Example: Gold tokens → collateral for stablecoin loans.

Competitor Analysis -

Several existing systems provide partial solutions but lack full compliance + ZK privacy + programmable ownership.

1. Digital Gold Platforms -
   Examples:
    - Paytm Gold
    - PhonePe Gold
    - MMTC-PAMP

   Characteristics

    - Users buy digital gold
    - Stored by custodian
    - Not blockchain-based

   Limitations:

    - No transparency
    - No programmability
    - No fractional DeFi utility
2. ETFs (Exchange Traded Funds) - like gold
   Advantages:
    - Regulated
    - Liquid

   Limitations:

    - Centralized
    - Limited transparency
    - No direct asset ownership
3. REITs (Real Estate Investment Trusts) - Used for property investment.

Advantages:

- Fractional ownership

Limitations:

- Limited liquidity
- Managed centrally
- No global access

4. Blockchain RWA Platforms -
   Examples:
   - RealT – real estate tokenisation
   - Centrifuge – tokenised assets for DeFi
   - Polymesh – security token infrastructure

   Limitations:

   - Complex compliance processes
   - Identity privacy issues
   - Limited retail accessibility

Feature vs Limitation Comparison Table -

Feature	BharatRWA	Digital Gold	ETFs	REITs	Existing RWA Platforms
Fractional ownership	✓	✓	✓	✓	✓
Blockchain transparency	✓	✗	✗	✗	✓
Programmable smart contracts	✓	✗	✗	✗	✓
Zero Knowledge KYC	✓	✗	✗	✗	✗
Global liquidity	✓	✗	Limited	Limited	Limited
DeFi integration	✓	✗	✗	✗	Partial
Custodian verification	✓	✓	✓	✓	✓

Privacy preserving compliance	✓	✗	✗	✗	✗

Project Feasibility Analysis -

Technical Feasibility -

The proposed system is technically feasible due to the maturity of modern blockchain development tools.

The project uses the following technologies:

Component	Technology
Blockchain network	Ethereum Sepolia
Smart contract language	Solidity
Development framework	Hardhat
Security libraries	OpenZeppelin
ZK Proof System	Noir
ZK Circuit Framework	Noir
Backend server	Node.js
Database	PostgreSQL
Oracle infrastructure	Chainlink
Frontend	Next.js + Ethers.js

Economic Feasibility -

The proposed system reduces operational costs compared to traditional financial infrastructures.

Advantages include:

- Automation of compliance processes using smart contracts
- Reduced reliance on intermediaries such as brokers and clearing houses
- Lower transaction costs through blockchain-based settlement
- Fractional investment enabling broader investor participation

By eliminating manual verification and settlement processes, the system significantly reduces operational overhead.

Operational Feasibility -

The system is designed to integrate with existing financial and asset custody infrastructures.

Operational processes include:

1. Custodian verification of physical assets
2. Asset registration on blockchain
3. Investor onboarding via Zero-Knowledge KYC
4. Token distribution representing fractional ownership
5. Dividend distribution based on token holdings

Since the user interface is web-based and integrates with wallets like **MetaMask**, the system remains accessible to users without requiring specialized technical knowledge.

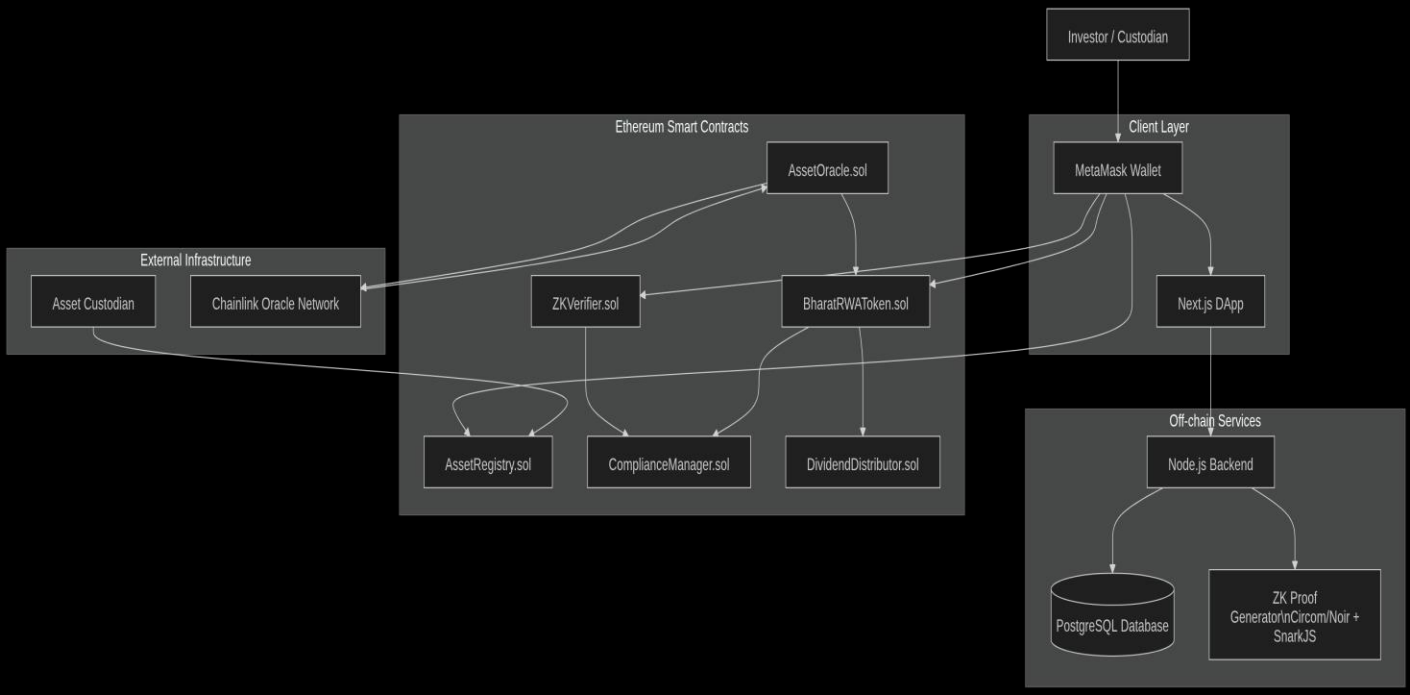
System Architecture -

The BharatRWA system uses a multi-layer blockchain architecture.

Main Layers:

- User Layer
- Application Layer
- Backend Layer
- Blockchain Layer
- Asset Layer

Architecture Diagram -



Layer Explanation-

1. User Layer Actors: They interact using wallets.
   - Investors
   - Asset custodians
   - Administrators
2. Application Layer:
   Frontend built using:
   - Next.js
   - Ethers.js
   - MetaMask

   Responsibilities:

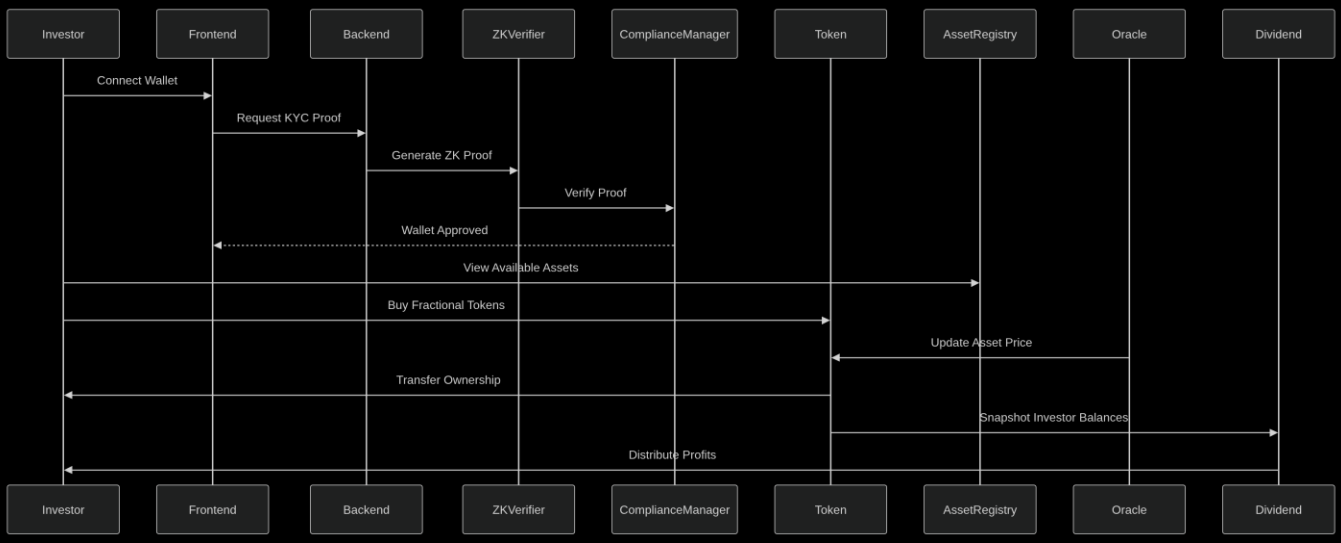
   - User interface
   - Wallet integration
   - Investment dashboard
3. Backend Layer: Backend manages off-chain operations.
   Components:
   - Node.js API
   - PostgreSQL database
   - ZK proof generation

Responsibilities:

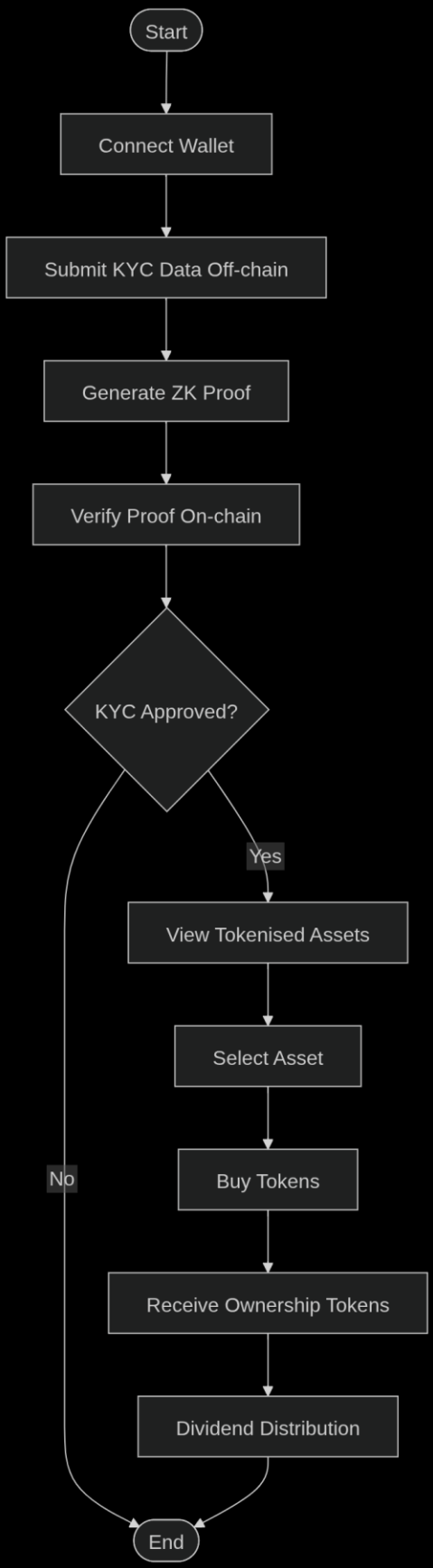
- Identity verification
- Asset metadata storage
- Compliance checks

4. Blockchain Layer: Contains smart contracts.
   Responsibilities:
   - Asset registration
   - Token minting
   - Compliance enforcement
   - Profit distribution
5. Asset Layer: Represents physical assets stored by custodians.
   Examples: Gold vaults, Real estate holdings. Custodians provide proof of reserve.

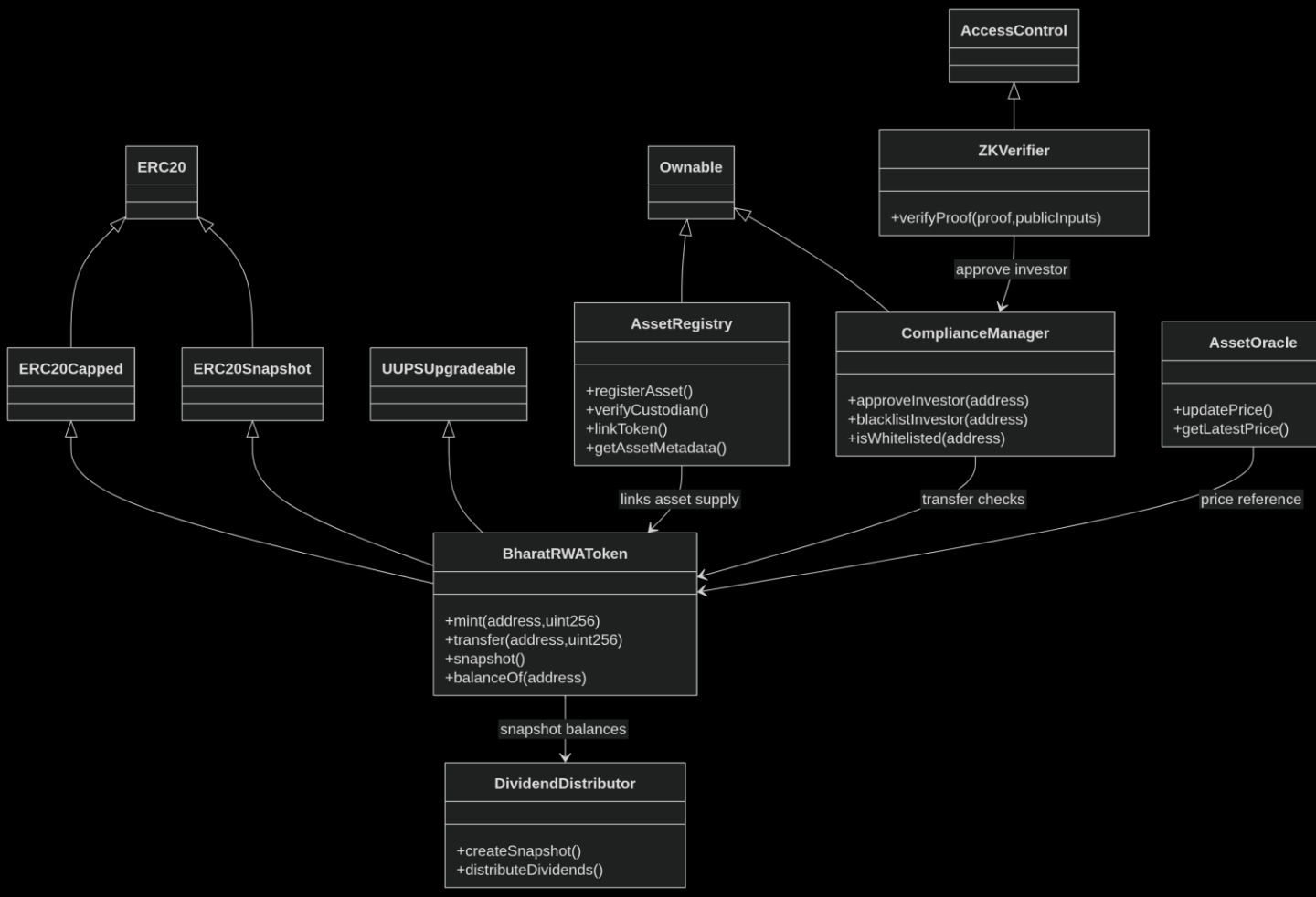
Process Flow Diagram :



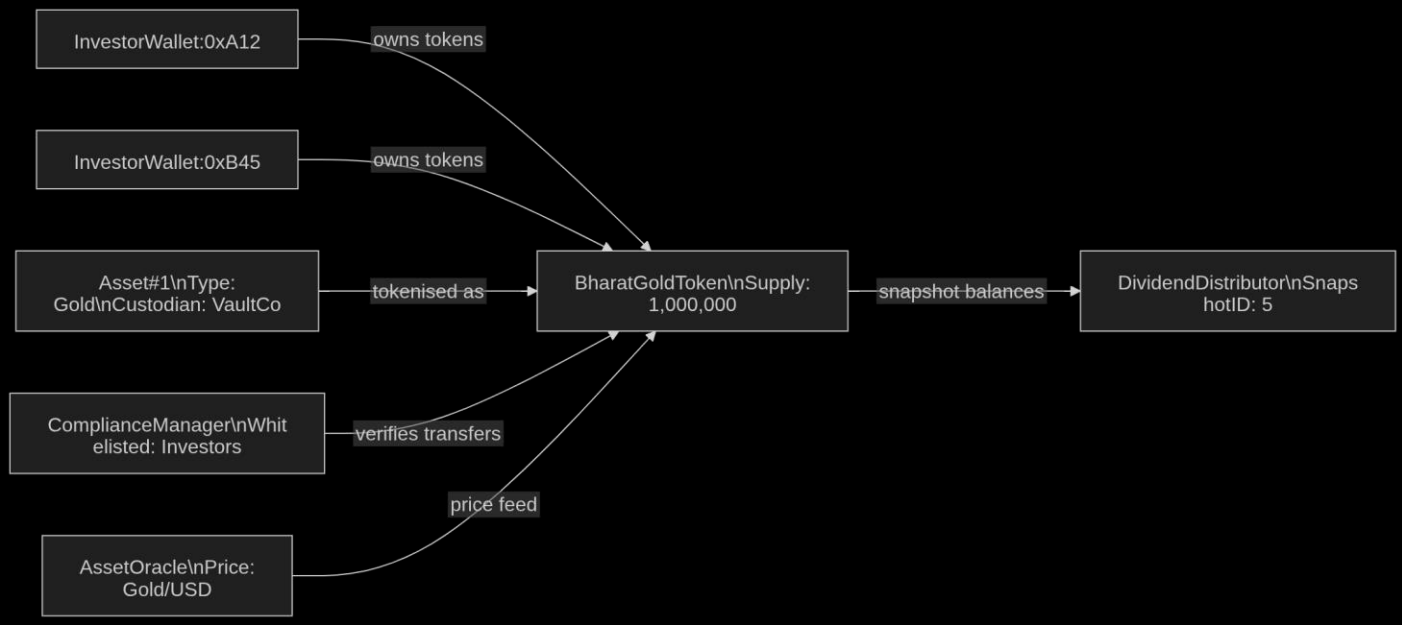
Flowchart of User Investment Process :


Start
Connect Wallet
Submit KYC Data Off-chain
Generate ZK Proof
Verify Proof On-chain
KYC Approved?
Yes
No
View Tokenised Assets
Select Asset
Buy Tokens
Receive Ownership Tokens
Dividend Distribution
End

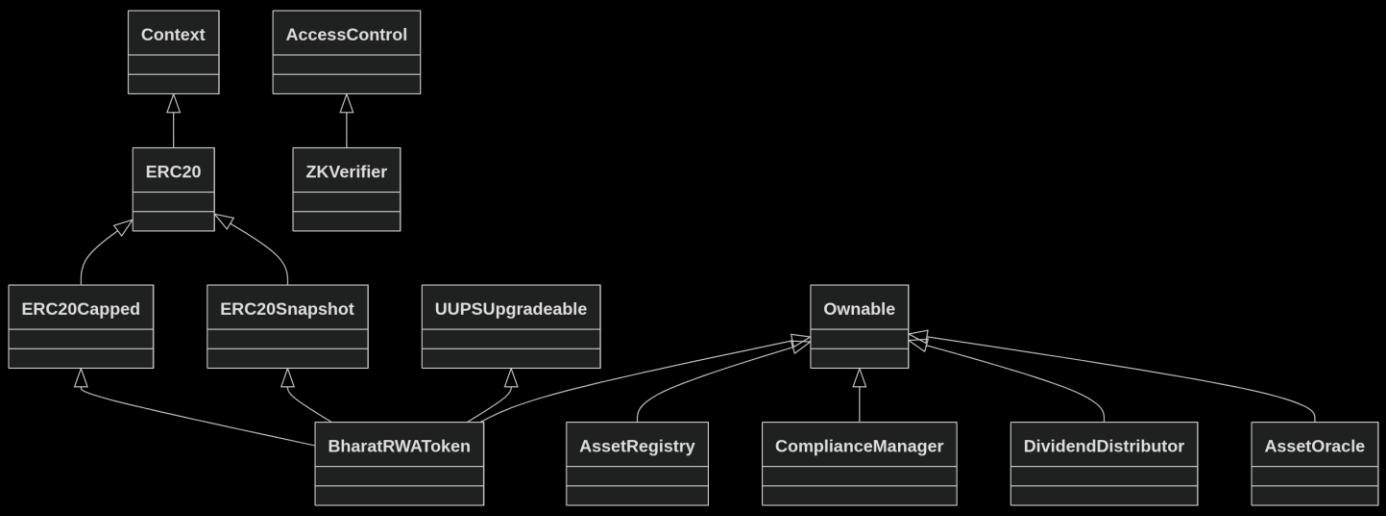
Class Diagram (Smart Contracts):



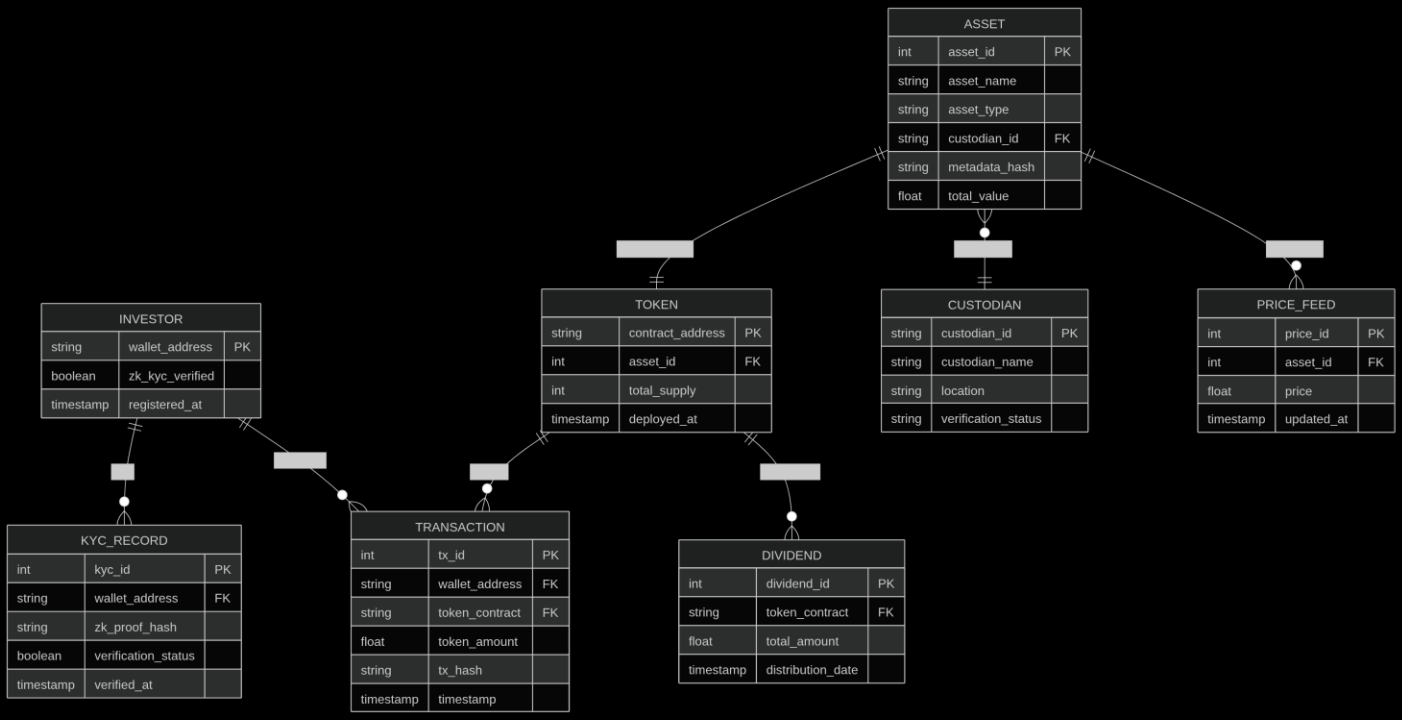
Object Diagram:



Inheritance Diagram:



ER Diagram(off-chain data):



Objectives of the Proposed System:

The primary objective of this project is to develop a secure, transparent, and compliant infrastructure for tokenisation assets of real-world using blockchain technology.

Specific Objectives

1. To enable fractional ownership of real-world assets through blockchain tokenisation.
2. To implement smart contract-based compliance mechanisms that enforce regulatory requirements automatically.
3. To develop a privacy-preserving identity verification system using Zero-Knowledge Proofs.
4. To create an on-chain asset registry that links digital tokens with real-world assets verified by custodians.
5. To integrate real-time asset price feeds using oracle services.
6. To implement a dividend distribution mechanism using token balance snapshots.
7. To design a secure and scalable architecture combining on-chain and off-chain components.

Relevance of Algorithms and Techniques:

The proposed system uses several advanced blockchain and cryptographic techniques to ensure security, scalability, and privacy.

1. ERC20 Token Standard -
   Advantages:
   - standardized token interface
   - interoperability with wallets and exchanges
   - programmable transfer logic
2. Zero-Knowledge Proofs -
   Zero-Knowledge Proofs allow verification of information without revealing the underlying data.
   In this system, Groth16 proofs are used to verify that:
   - an investor satisfies KYC requirements
   - without revealing personal identity details on-chain.

   Advantages:

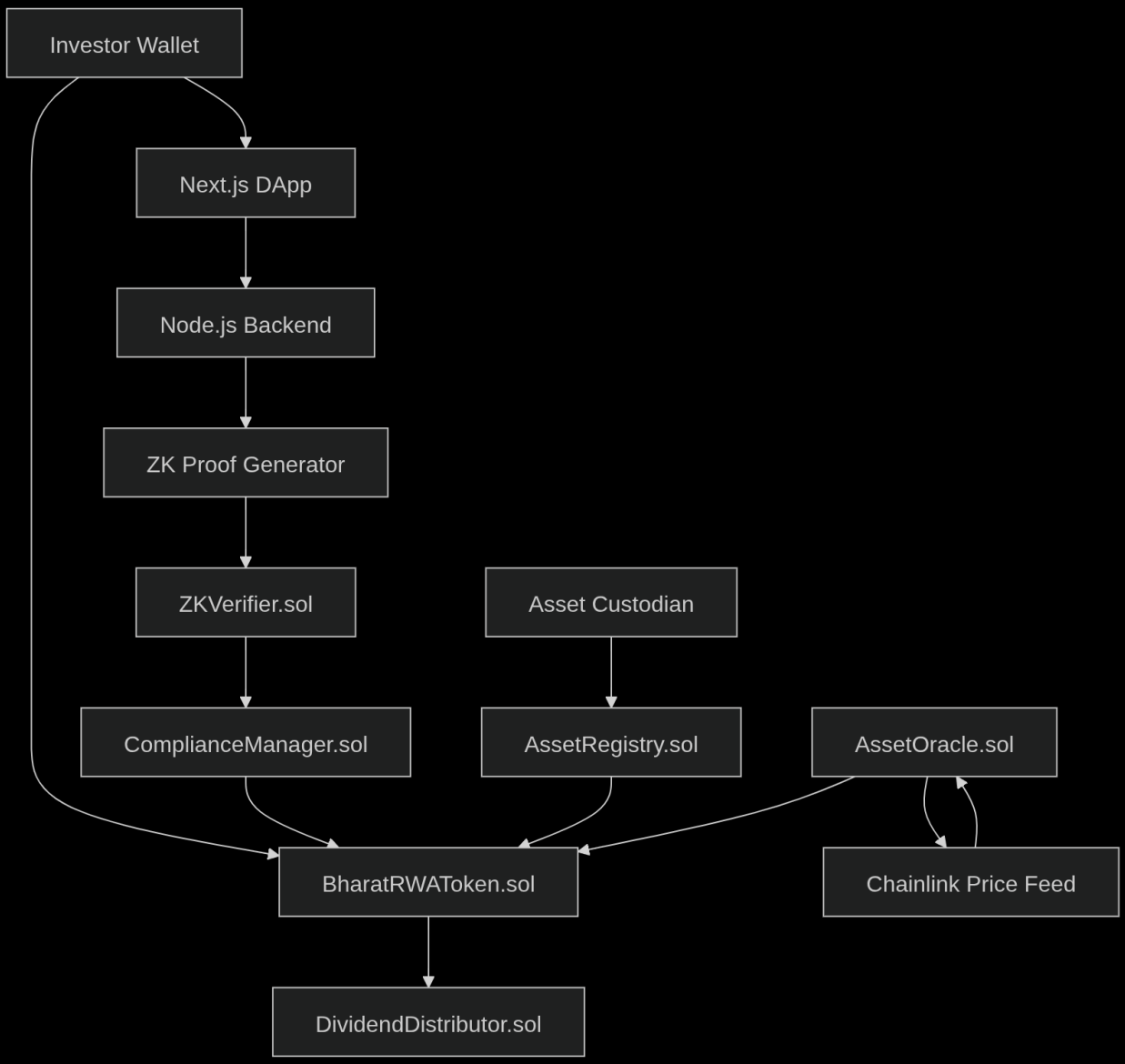
   - privacy preservation
   - regulatory compliance
   - efficient verification on Ethereum
3. Snapshot Mechanism -
   The snapshot mechanism records token balances at a specific block number. This technique is used in the DividendDistributor contract.
   Purpose:
   - determine investor holdings at the time of profit distribution
   - prevent manipulation of balances during dividend payouts
4. Oracle-Based Price Feeds -
   Real-world asset prices cannot be directly accessed by smart contracts. Therefore, the system uses oracle services (Chainlink) to fetch external asset price data.
   Advantages:
   - reliable off-chain data integration
   - tamper-resistant price feeds
   - decentralized oracle infrastructure
5. Smart Contract Access Control -
   Access control mechanisms are implemented using OpenZeppelin libraries such as:
   - Ownable
   - AccessControl

These mechanisms restrict sensitive operations such as:

- token minting
- asset registration
- compliance updates

This ensures system security and prevents unauthorized actions.

Smart Contract Interaction Architecture:



Algorithms and Pseudocode:

1. Asset Registration Algorithm :
   Input: assetDetails, custodianID, metadataHash, tokenSupply
   Output: assetID

   1. Verify custodian authenticity
   2. If custodian is not verified
      Reject registration
   3. Generate unique assetID
   4. Store assetDetails, metadataHash, custodianID in AssetRegistry
   5. Deploy BharatRWAToken contract
   6. Set tokenSupply as maximum cap
   7. Link token contract address with assetID
   8. Emit AssetRegistered event
   9. Return assetID

2. Zero-Knowledge KYC Verification Algorithm:

Input: investorWallet, zkProof, publicInputs
   Output: verificationStatus
   1. Receive zkProof from investor
   2. Send proof to ZKVerifier contract
   3. Call verifyProof(zkProof, publicInputs)

   4. If proof is valid
      Call ComplianceManager.approveInvestor(investorWallet)
      Set verificationStatus = TRUE
   Else
      Reject verification
      Set verificationStatus = FALSE

   5. Emit KYCVerified event
   6. Return verificationStatus

3. Token Purchase Algorithm:

Input: investorWallet, tokenAmount

Output: transactionStatus

1. Check if investorWallet is KYC approved
2. If investor not approved

    Reject transaction
3. Check if investorWallet is blacklisted
4. If blacklisted

    Reject transaction
5. Check token availability
6. If tokenAmount exceeds remaining supply

    Reject transaction
7. Transfer tokens to investorWallet
8. Update investor balance
9. Emit TokenPurchased event
10. Return transactionStatus = SUCCESS

4. Token Transfer with Compliance Check:

Input: senderWallet, receiverWallet, amount
Output: transferStatus
1. Check ComplianceManager for senderWallet approval
2. Check ComplianceManager for receiverWallet approval
3. If either wallet is not approved

    Reject transfer
4. Check sender balance
5. If balance < amount

    Reject transfer
6. Execute token transfer
7. Update balances

8. Emit Transfer event
9. Return transferStatus = SUCCESS

5. Dividend Distribution Algorithm:
   Input: totalDividendAmount

   Output: distributionStatus

   1. Create snapshot of token balances

   2. Retrieve all token holders from snapshot

   3. For each holder in tokenHolders

      holderShare = holderBalance / totalTokenSupply

      dividendAmount = holderShare * totalDividendAmount

      Transfer dividendAmount to holder wallet

   4. Emit DividendDistributed event

   5. Return distributionStatus = SUCCESS

6. Oracle Price Update Algorithm:

   Input: assetID, latestPrice
   Output: updateStatus
   1. Fetch latest asset price from Chainlink oracle
   2. Validate oracle response
   3. If response invalid
      Reject update
   4. Update asset price in AssetOracle contract
   5. Store timestamp of update
   6. Emit PriceUpdated event
   7. Return updateStatus = SUCCESS

7. Investor Investment Workflow Algorithm:
   Input: investorWallet

   Output: investmentStatus

   1. Investor connects wallet to DApp

2. Investor submits KYC data off-chain

3. Backend generates Zero-Knowledge proof

4. ZKVerifier validates proof on-chain

5. If KYC verification fails

Stop process

6. Investor selects asset

7. Investor chooses token amount

8. System verifies compliance rules

9. Execute token purchase transaction

10. Record transaction in database

11. Return investmentStatus = SUCCESS

Sources : https://www.coursera.org/learn/regtech/